

Customer Churn Prediction



A Walkthrough of the Analysis, Modelling and Reasoning Behind It

THANVEER AHMED T M

Dataset: Telco Customer Churn (Kaggle)

Tools: Python, Pandas, NumPy, Matplotlib, Seaborn, Plotly, Scikit-learn

1. What This Project Is About

Telecom companies lose a meaningful slice of their customer base every month to churn — customers who cancel their subscription. Winning back a lost customer, or acquiring a new one, generally costs more than keeping an existing one happy. So the practical question behind this project is simple to state and harder to answer: which customers are quietly on their way out, and what do they have in common?

The approach taken here follows the shape most analytics work takes — first understand and clean the raw data, then explore it visually to see what actually separates customers who leave from customers who stay, then prepare that data for a model, then try a handful of different modelling approaches and judge them honestly, not just by a single accuracy number.

2. About the Dataset

The dataset holds roughly 7,000 customers from a telecom provider, one row per customer. Alongside basic demographics, it records which services each person subscribes to, how they're billed, and — the column everything else is measured against — whether they churned.

2.1 The columns, grouped by what they describe

- Who the customer is: gender, SeniorCitizen, Partner, Dependents
- How long they've stayed and under what terms: tenure, Contract
- What they've subscribed to: PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies
- How they're billed: PaperlessBilling, PaymentMethod, MonthlyCharges, TotalCharges
- The outcome we care about: Churn (Yes/No)

A quick note on how a couple of these relate to each other, since it comes up more than once later: TotalCharges is roughly tenure multiplied by MonthlyCharges, so a customer's cumulative bill is really just a reflection of how long they've stuck around and how much they pay per month.

3. Cleaning the Data Before Trusting It

3.1 TotalCharges was hiding as text

On first inspection, TotalCharges looked numeric but was actually stored as a string column. The cause turned out to be a handful of rows containing a blank space instead of a number — and a single non-numeric entry is enough to make pandas treat the whole column as text.

```
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
```

pd.to_numeric converts the column to real numbers, and errors='coerce' tells it to turn anything it can't parse into NaN rather than crashing the whole operation. This step matters because nearly everything downstream — plotting, scaling, correlation, modelling — assumes numeric columns behave like numbers.

3.2 A small number of genuinely missing values

Once TotalCharges was numeric, a handful of rows showed up as missing. Checking those rows revealed a simple explanation: every one of them had tenure equal to zero — brand-new customers who hadn't been billed yet, so there was nothing to report.

```
df.drop(labels=df[df['tenure'] == 0].index, axis=0, inplace=True)
```

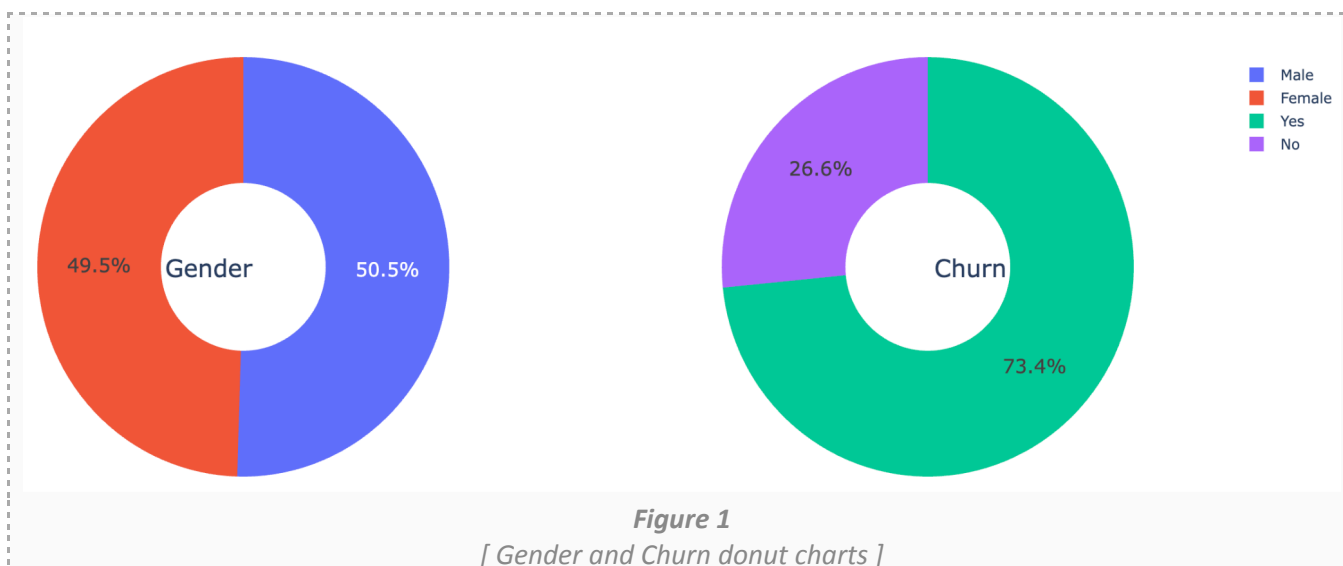
Since there's no sensible number to fill in for someone who joined this month, and since this only affected around eleven rows out of roughly seven thousand, those rows were simply removed rather than guessed at.

4. Exploring the Data for Patterns

Before building anything predictive, the data was explored visually — the point being to build an honest, evidence-based sense of what actually relates to churn, rather than guessing.

4.1 Gender and churn

Two donut charts compared the overall split of Gender and Churn side by side, built with Plotly's Pie chart and a hollow centre.

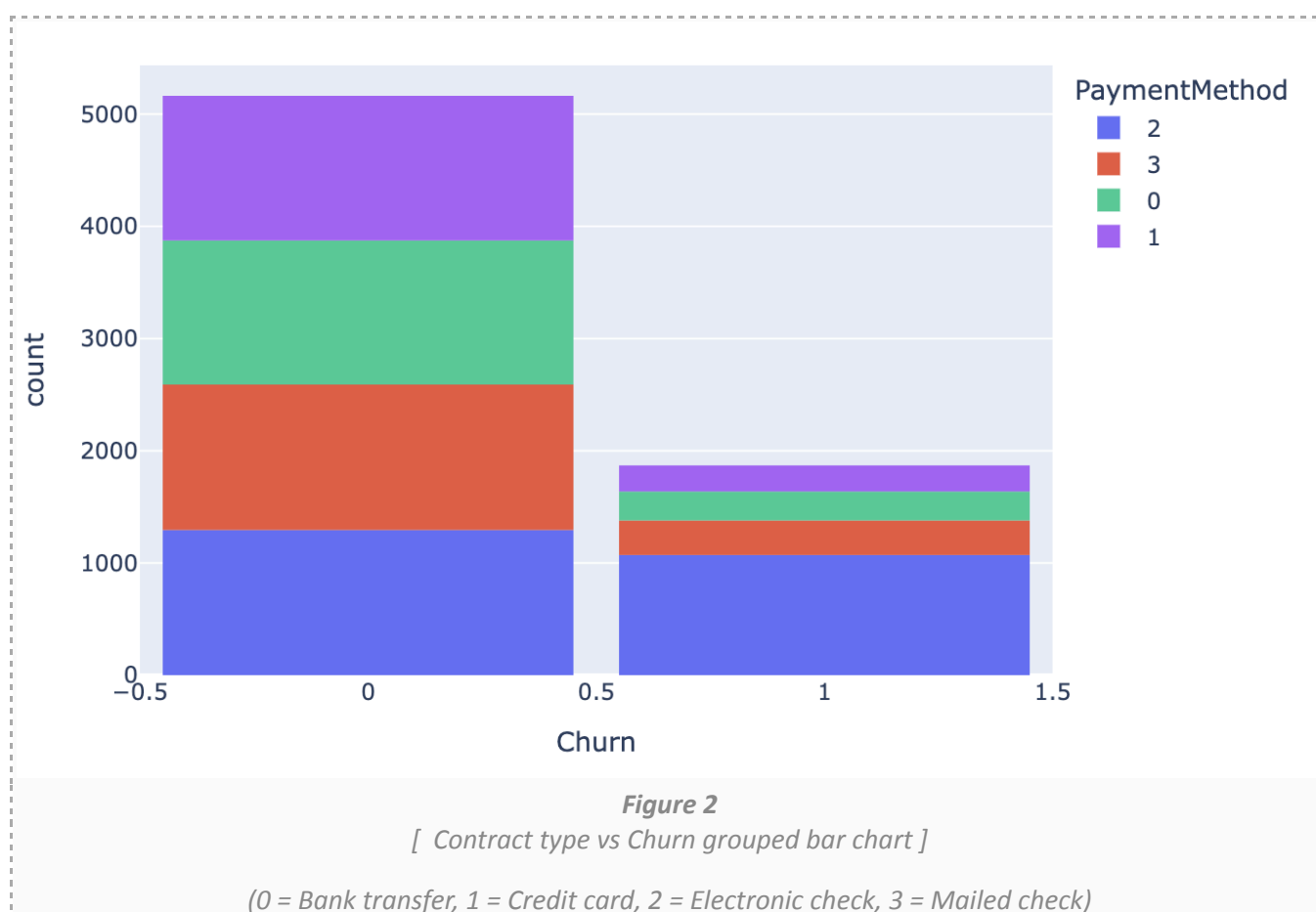


Gender came out almost perfectly balanced — 49.5% female, 50.5% male — and showed no visible connection to churn. The overall churn rate across the whole dataset sat at 26.6%, which became the baseline every other finding was measured against.

4.2 Contract type and churn

A grouped bar chart compared Churn against Contract type, letting Plotly count rows directly from the raw data rather than pre-tallying anything by hand.

```
px.histogram(df, x='Churn', color='Contract', barmode='group')
```



The pattern here was the clearest of the whole exploration: customers on month-to-month contracts churn far more often than customers locked into a one- or two-year term. This turned out to be one of the strongest signals in the entire dataset.

4.3 Billing amounts and churn

Kernel density plots compared the shape of MonthlyCharges and TotalCharges between customers who churned and those who didn't, using a smoothed curve rather than raw bars so that two groups of very different sizes (26.6% versus 73.4%) could still be compared fairly.

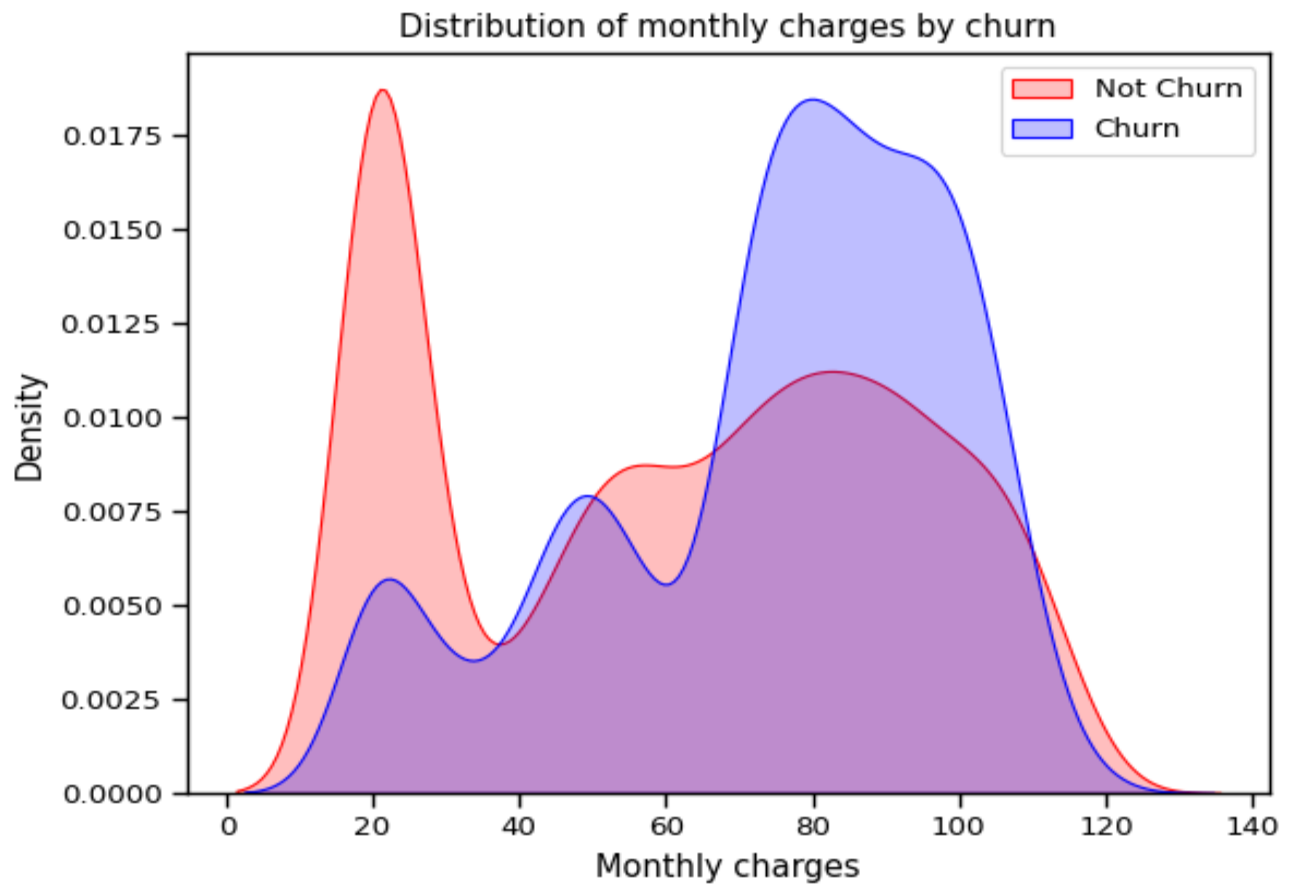
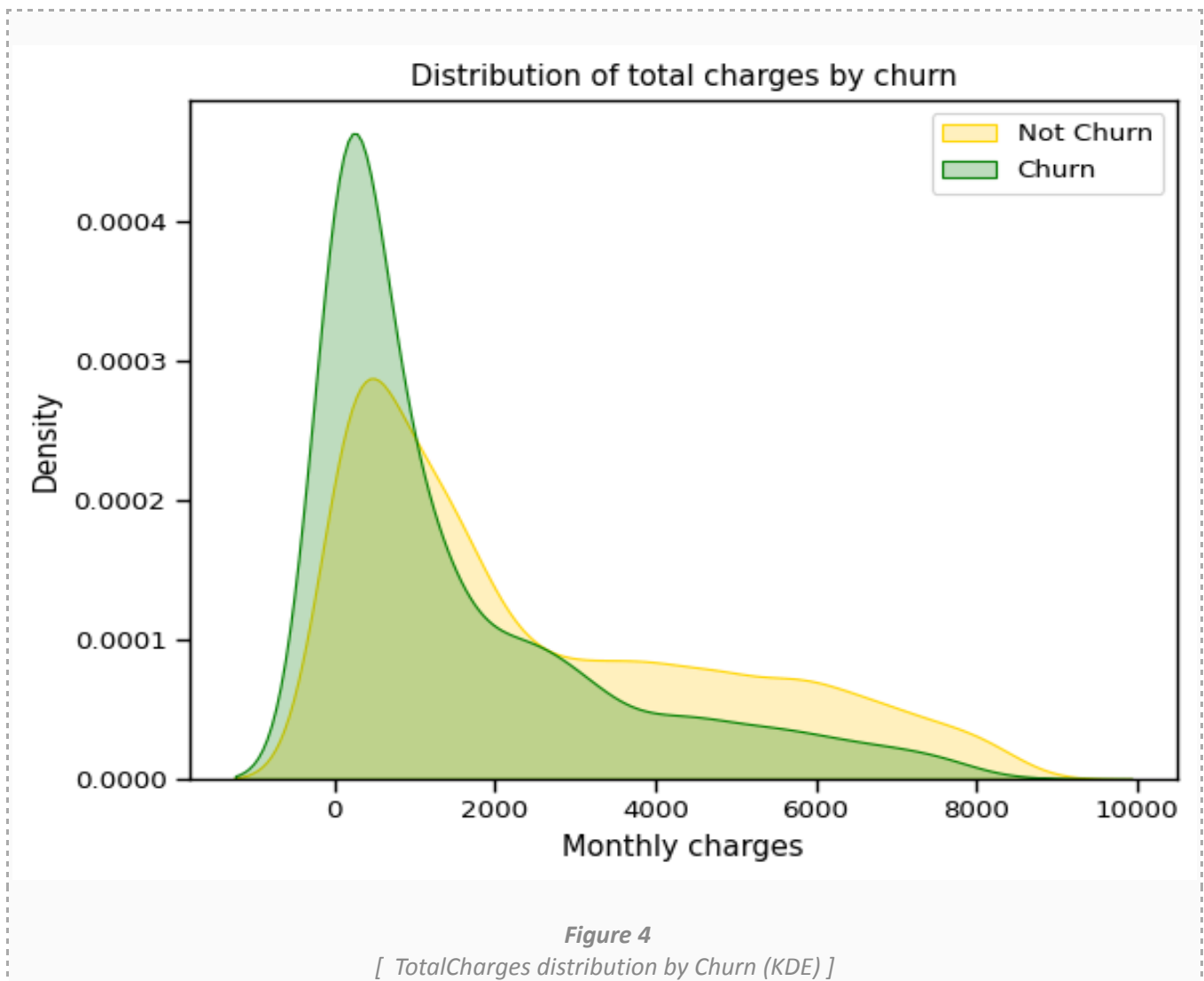


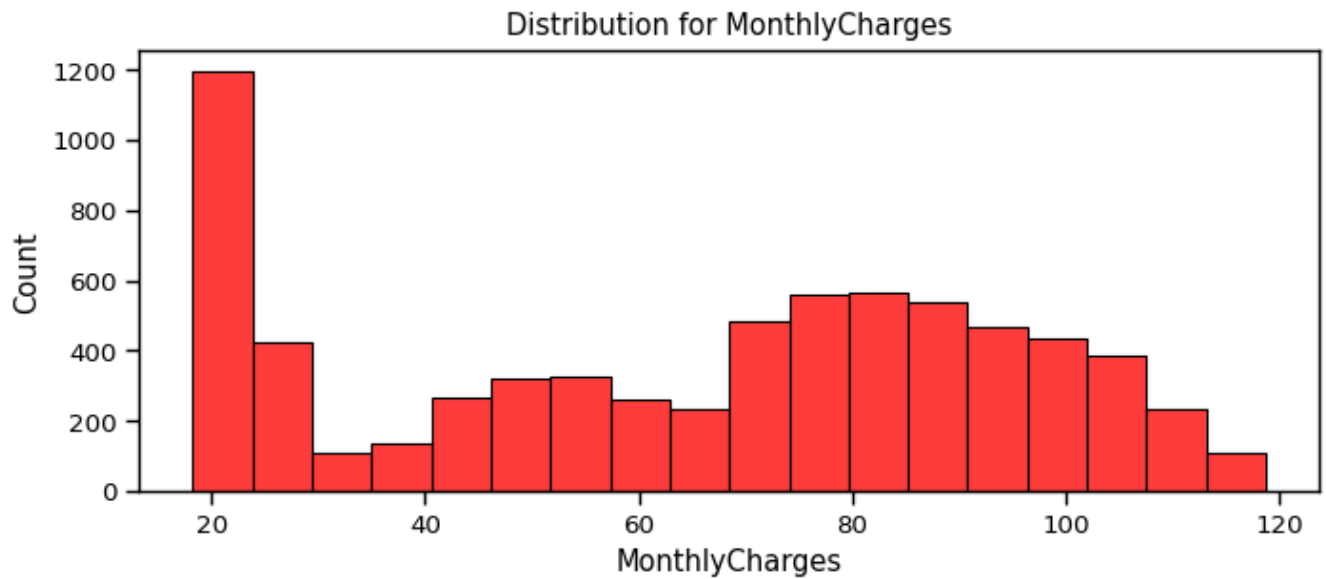
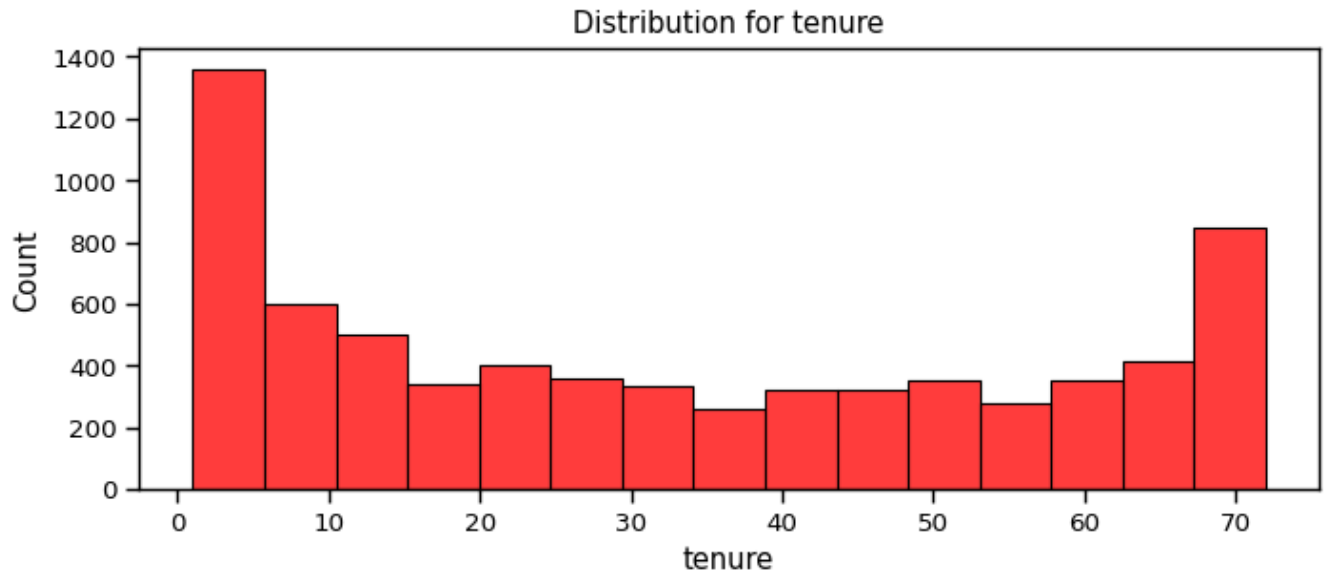
Figure 3

[MonthlyCharges distribution by Churn (KDE)]



Customers who churned tended to pay more per month than customers who stayed. At the same time, they clustered at low TotalCharges — which makes sense once you notice they also tend to leave early, before their bill has had time to add up.

4.4 The shape of tenure, MonthlyCharges and TotalCharges on their own



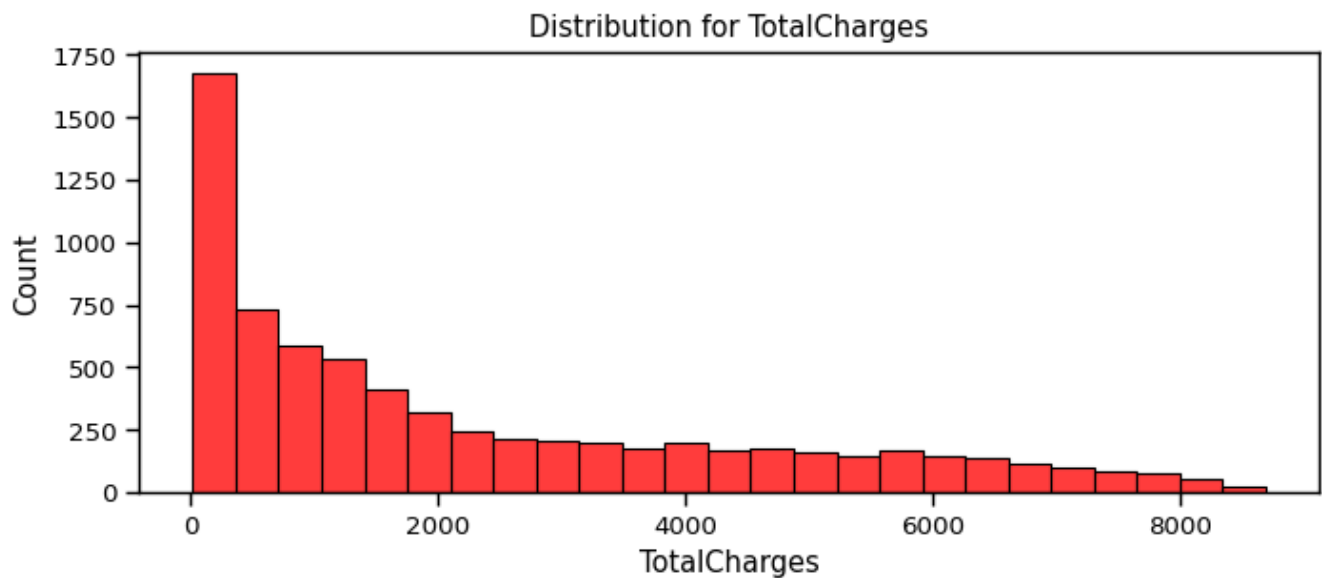


Figure 5
[Histograms of tenure, MonthlyCharges and TotalCharges]

Tenure showed a U-shape: a large cluster of very new customers, a smaller cluster of long-standing ones, and comparatively few in between. MonthlyCharges had a spike at the cheap end (likely phone-only customers) and a broader hump higher up (likely bundled internet and add-ons). TotalCharges was heavily right-skewed, which follows naturally from so many customers being new.

4.5 Internet service, gender, and churn together

A grouped bar chart with a two-level axis (Churn and Gender together) compared churn across the three InternetService categories.

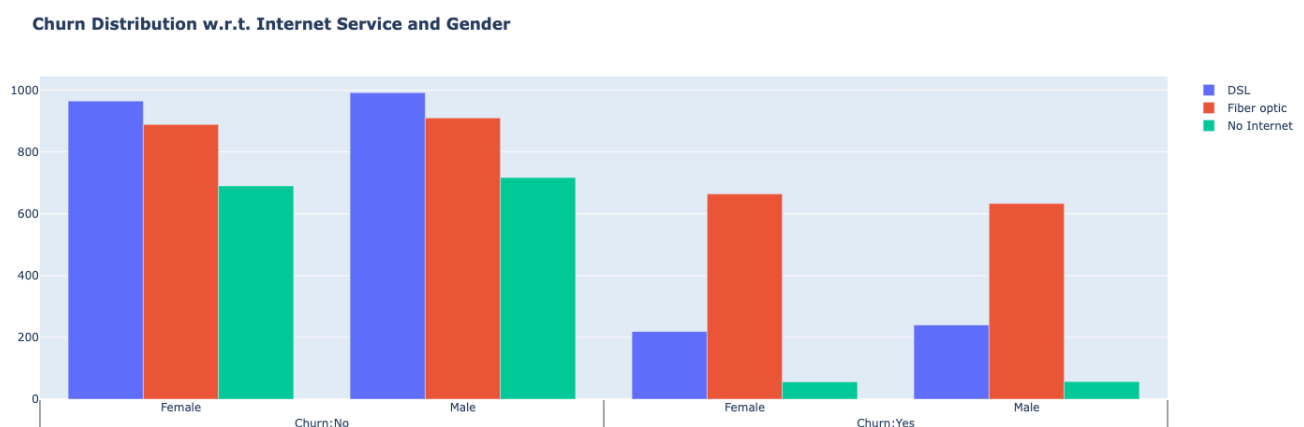


Figure 6
[Churn by Internet Service and Gender]

Fiber optic customers churned noticeably more than DSL or no-internet customers, and this held regardless of gender — reinforcing that InternetService type carries real signal, while gender, again, does not.

4.6 Add-on services and churn

Customers without OnlineSecurity churned more than those who had it — plausibly because subscribing to an add-on reflects a customer who is more invested in the service overall, not just paying for the bare minimum.

4.7 A correlation view across every column

Since correlation only works on numbers, every column — including the text ones — was first converted to numeric codes, and a full correlation matrix was calculated and drawn as a heatmap, with the duplicate half of the grid hidden for clarity.

```
corr = data.apply(lambda x: pd.factorize(x)[0]).corr()
```

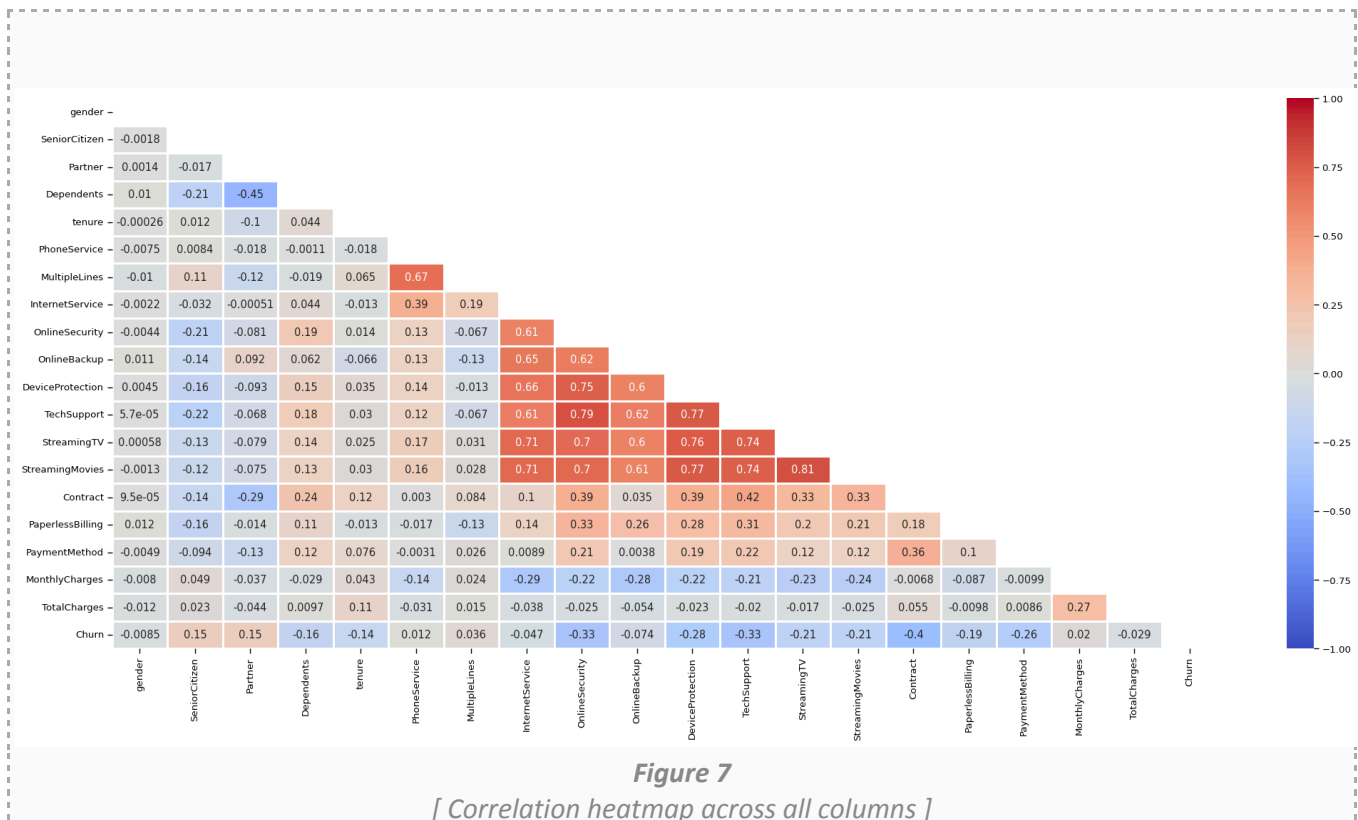


Figure 7
[Correlation heatmap across all columns]

This heatmap mostly confirmed what the individual charts had already shown — Contract and tenure stood out as the columns most related to churn, while gender sat near zero — but it did so in one consolidated view rather than chart by chart.

5. Checking for Outliers

Before any numbers get fed into a model, it's worth checking whether any of them are extreme enough to be data-entry mistakes, or simply unusual enough to unfairly pull an average in one direction.

- Q1, Q2 (median) and Q3 split the sorted data into quarters.
- IQR (interquartile range) = Q3 minus Q1 — the width of the middle half of the data, deliberately ignoring the extremes.
- Lower fence = $Q1 - 1.5 \times IQR$, upper fence = $Q3 + 1.5 \times IQR$.

Anything outside those two fences gets flagged as an outlier.

```
Q1 = df['MonthlyCharges'].quantile(0.25)
Q3 = df['MonthlyCharges'].quantile(0.75)
IQR = Q3 - Q1
lower_fence = Q1 - 1.5 * IQR
upper_fence = Q3 + 1.5 * IQR
```

In this dataset the numeric columns turned out to be reasonably well-behaved, but the check itself is a standard habit worth keeping regardless of the outcome — it's cheap to run and catches real problems when they exist.

6. Getting the Data Model-Ready

6.1 Putting the numeric columns on the same scale

tenure, MonthlyCharges and TotalCharges live on very different scales — roughly 0 to 72, 18 to 120, and 0 to 8,700. Several algorithms used later (KNN, SVM, Logistic Regression) are sensitive to this: a column with naturally larger numbers can end up dominating the model's calculations even when it isn't actually more important.

```
scaler = StandardScaler()
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])
```

StandardScaler rescales each column so it has a mean of zero and a standard deviation of one, using the formula (value minus mean) divided by standard deviation — standard deviation being a measure of how spread out the values are. Fit_transform is used only on the training data, which both calculates the scaling rule and applies it. The test data only ever goes through transform, reusing the exact rule learned from training rather than calculating a fresh one. This matters because the test set is meant to stand in for future, unseen customers — and in real use, you can't recalculate a mean and standard deviation from a customer who hasn't arrived yet.

6.2 Turning categories into numbers

Every model needs numbers, not words, so the remaining text columns were converted using two different methods depending on the column:

- One-hot encoding for columns with several unordered categories — PaymentMethod, Contract, InternetService — where turning each category into its own yes/no column avoids implying a ranking between options that don't actually have one.
- Label encoding for the mostly binary yes/no columns — gender, Partner, Dependents, and the rest — where a simple 0/1 code carries no such risk.

```
cat_cols_ohe = ['PaymentMethod', 'Contract', 'InternetService']
cat_cols_le = list(set(X_train.columns) - set(num_cols) - set(cat_cols_ohe))
```

The second line collects whatever columns are left over once the numeric and one-hot groups are removed, using simple set subtraction rather than typing out every remaining column name by hand.

7. Splitting the Data Before Modelling

```
X = df.drop(columns=['Churn'])
y = df['Churn'].values
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=40, stratify=y)
```

X holds every column except Churn — the clues a model gets to see. y holds only Churn — the real answer, kept separate so the model is never handed the answer directly while learning.

The data is then split roughly 70/30. The larger portion (X_train, y_train) is what each model studies from. The smaller portion (X_test, y_test) is held back completely, untouched during training, and used afterwards purely to check whether a model actually learned something general — rather than simply memorising the training rows, the same way testing a student on questions they already know the answers to wouldn't tell you much.

Stratify=y keeps the same 73.4% / 26.6% churn ratio in both the training and test sets, so the comparison stays fair. random_state=40 just makes the split repeatable from one run to the next.

8. Trying Several Different Models

No single algorithm is guaranteed to be the best fit for a given dataset, so rather than committing to one upfront, several genuinely different approaches were trained and compared on equal footing.

8.1 K-Nearest Neighbors

```
knn_model = KNeighborsClassifier(n_neighbors=11)
knn_model.fit(X_train, y_train)
accuracy_knn = knn_model.score(X_test, y_test)
```

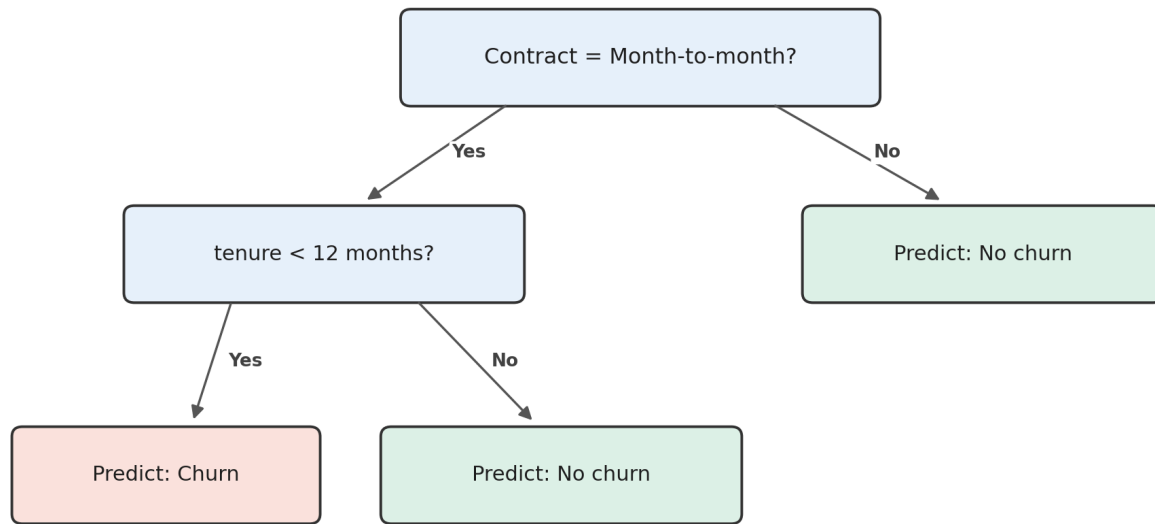
KNN predicts a new customer by finding the eleven most similar existing customers and going with whatever the majority of them did. It's intuitive, but it relies heavily on distance calculations, which is exactly why scaling the numeric columns beforehand mattered.

8.2 Decision Tree

```
dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)
accuracy_dt = dt_model.score(X_test, y_test)
```

A decision tree learns a sequence of yes/no questions from the training data — at every branch it tests every available column, measures how well each possible split separates churners from non-churners, and keeps whichever split works best. It's easy to interpret, but left unrestricted it tends to keep splitting until it has essentially memorised the training data, which hurts how well it generalises to new customers. This showed up clearly in the results: without any depth limit, this model performed noticeably worse than the others.

Decision Tree — Simplified Churn Prediction Example



Simplified example — the real tree tests every column at every branch and typically goes several levels deeper than shown here.

Figure 8
[Decision tree structure]

8.3 Random Forest

```
model_rf = RandomForestClassifier(n_estimators=500, oob_score=True,  
    n_jobs=-1, random_state=50, max_features='sqrt', max_leaf_nodes=30)  
model_rf.fit(X_train, y_train)
```

A random forest builds many decision trees — five hundred, in this case — each trained on a random slice of the data and features, then combines their votes. This ensemble approach smooths out the overfitting problem a single tree runs into, generally making it more reliable.

8.4 Support Vector Machine

```
svc_model = SVC(random_state=1)  
svc_model.fit(X_train, y_train)  
accuracy_svc = svc_model.score(X_test, y_test)
```

SVM looks for the clearest possible dividing boundary between churned and non-churned customers, based on the points closest to that boundary. A new customer is classified by which side of that boundary they land on.

8.5 Logistic Regression

```
lr_model = LogisticRegression()  
lr_model.fit(X_train, y_train)  
accuracy_lr = lr_model.score(X_test, y_test)
```

Despite the name, this is a classification method: it works out a probability of churn as a weighted combination of the input features, and labels anything above fifty percent as churn. It's simple, quick, and easy to interpret, which makes it a natural baseline to measure the more elaborate models against.

8.6 AdaBoost

```
ada_model = AdaBoostClassifier(n_estimators=100, random_state=42)
ada_model.fit(X_train, y_train)
```

AdaBoost takes a different route to combining trees than random forest does. Rather than training many trees independently and averaging them, it trains trees one after another in sequence — each new tree pays extra attention to whichever customers the previous trees got wrong, and the trees are combined with more trust given to the more accurate ones. In principle this sequential, error-focused approach can help with a class that's harder to predict, such as the smaller group of customers who actually churn.

8.7 Combining models: a voting classifier

```
clf1 = GradientBoostingClassifier()
clf2 = LogisticRegression()
clf3 = AdaBoostClassifier()
ecf1 = VotingClassifier(estimators=[('gbc', clf1), ('lr', clf2), ('abc', clf3)], voting='soft')
ecf1.fit(X_train, y_train)
predictions = ecf1.predict(X_test)
```

As a final step, three models built on genuinely different logic — Gradient Boosting, Logistic Regression and AdaBoost — were combined into a single voting classifier, rather than relying on any one of them alone. With soft voting, each model outputs a probability of churn for a given customer, and those probabilities are averaged together into one final decision. The reasoning is that different models tend to make different mistakes; combining several can smooth out an individual model's blind spots, provided the models being combined don't all think in the same way to begin with — which is why the three chosen here use noticeably different underlying logic rather than three variations on the same idea.

9. Judging the Models Properly

Accuracy alone can be quietly misleading here. Since only 26.6% of customers actually churn, a model that always predicted "No churn" would already score close to 73% accuracy without having learned anything useful. A few additional checks were used to see past that.

9.1 Precision, recall and F1 for each class

```
predictions = model.predict(X_test)
print(classification_report(y_test, predictions))
```

This breaks performance down separately for the two classes rather than blending them into one number:

- Precision — of everyone the model flagged as a churn risk, how many actually churned (a measure of false alarms).
- Recall — of everyone who actually churned, how many the model actually caught (a measure of missed cases).
- F1-score — a single figure balancing precision and recall together.

For Logistic Regression, overall accuracy landed at 81%, but recall for the churn class was only 58% — meaning 42% of customers who actually left were missed entirely. That's an important caveat: a respectable headline accuracy figure was quietly hiding a real weakness in catching the customers the business most wants to identify. The Decision Tree performed worse still (73% accuracy, 53% recall for churn), while AdaBoost landed close to Logistic Regression (81% accuracy, 55% recall), trading slightly fewer false alarms for slightly more missed churners.

9.2 Confusion matrix

```
sns.heatmap(confusion_matrix(y_test, predictions), annot=True, fmt='d')
```

The confusion matrix is the set of raw counts that the precision and recall figures above are calculated from — correctly predicted stays, correctly predicted churns, customers wrongly flagged as a churn risk, and, usually the most important number for the business, real churners the model missed entirely.

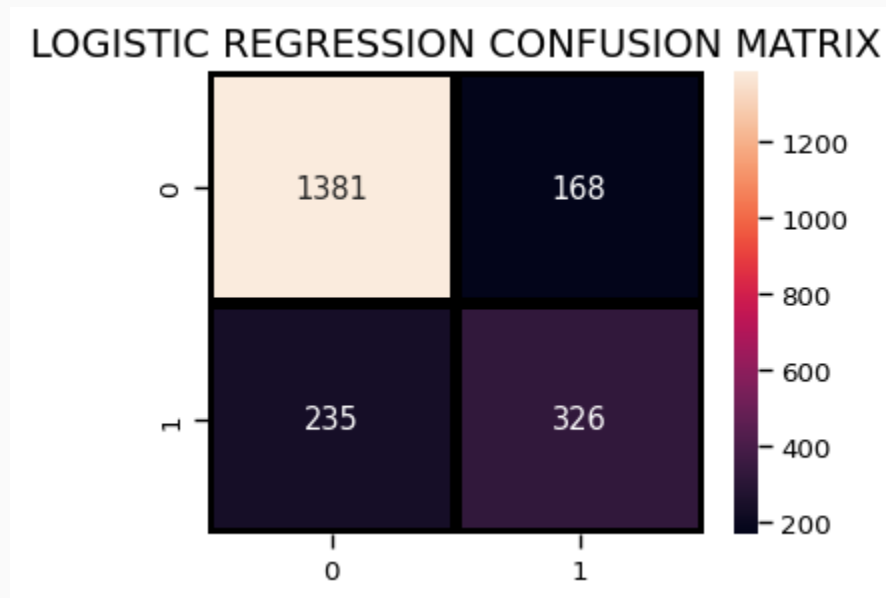


Figure 9
[Confusion matrix — Logistic Regression]

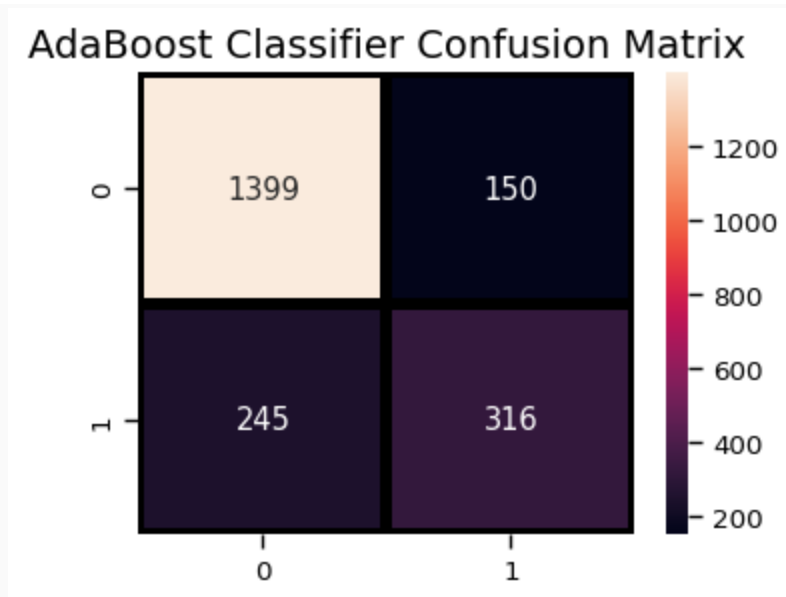


Figure 10
[Confusion matrix — AdaBoost]

For Logistic Regression, 234 real churners were missed out of 561. For AdaBoost, that number was 245, alongside slightly fewer false alarms than Logistic Regression produced. Neither model is clearly superior once both figures are weighed together — the better choice depends on whether the business cares more about catching every possible churner or avoiding unnecessary false alarms.

9.3 ROC curve

```
y_pred_prob = model.predict_proba(X_test)[:,-1]
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
plt.plot([0,1],[0,1], 'k--')
plt.plot(fpr, tpr)
```

Rather than judging a model at a single fixed cutoff, the ROC curve tests many different probability thresholds and plots how the true positive rate (churners correctly caught) trades off against the false positive rate (loyal customers wrongly flagged) at each one. The dashed diagonal line represents a model with no real skill; a curve that bulges toward the top-left corner represents a model doing a genuinely good job of separating the two groups.

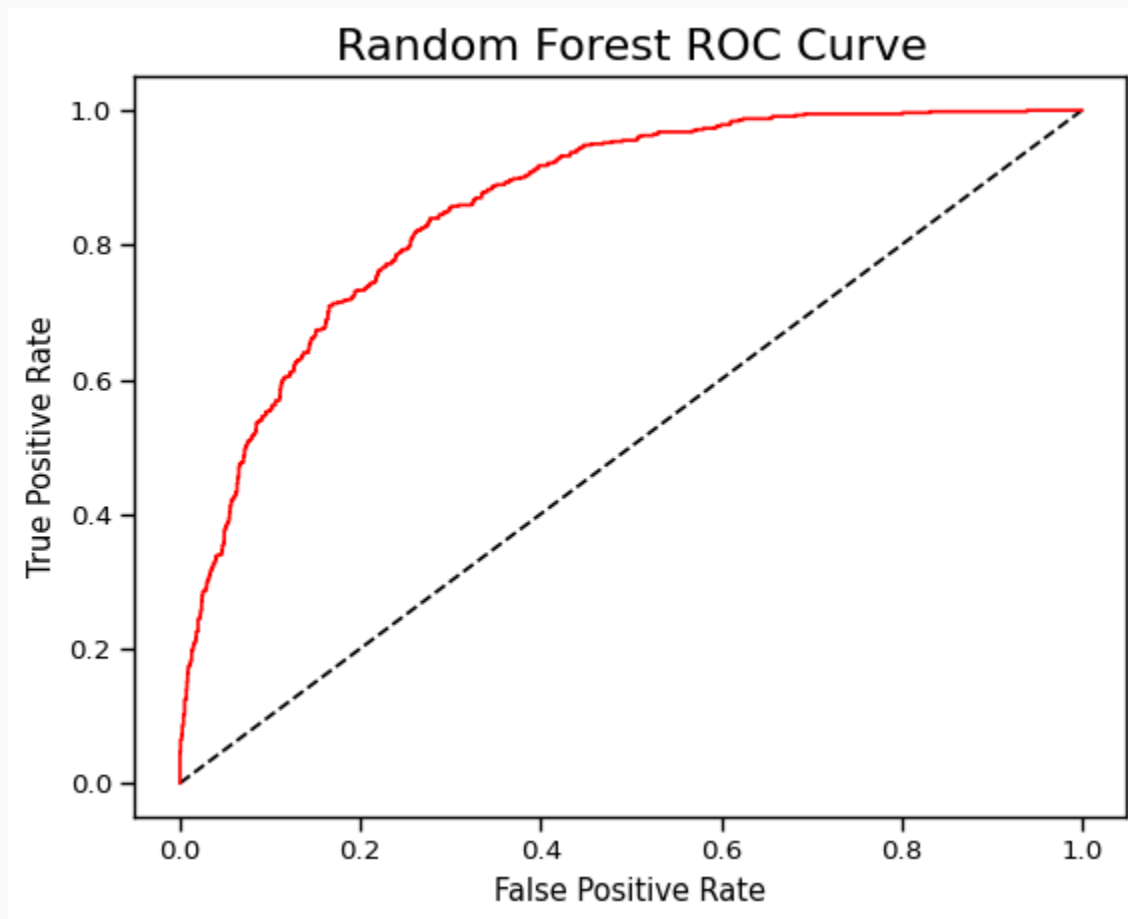


Figure 11
[ROC curve — Random Forest]

10. Putting It All Together

Step	What it was for
Data cleaning	Fix data types and handle a small number of missing or invalid values, so everything downstream can be trusted.
Exploration (charts)	Find out, with evidence, what actually relates to churn — before assuming anything.
Outlier check	Catch extreme or mistaken values that could otherwise distort averages or training.
Scaling	Put the numeric columns on a comparable footing so no single one unfairly dominates distance-based models.
Encoding	Turn categories into numbers a model can use, without implying rankings that don't exist.
Train/test split	Evaluate every model on data it has never seen, for an honest read of real-world performance.

Multiple models	Compare genuinely different approaches empirically, instead of assuming which one would win.
Deeper evaluation	Look past plain accuracy, given the imbalance between churners and non-churners, to see how well each model catches the customers that matter.

What the data says about churn

- Customers on month-to-month contracts churn far more than those on longer commitments — the single strongest pattern found.
- New customers are at noticeably higher risk than long-tenured ones.
- Higher monthly bills are associated with more churn, particularly among Fiber optic subscribers.
- Customers without add-on services such as Online Security churn more than those with them.
- Gender shows essentially no relationship with churn, at any point in the analysis.

What the modelling says

- Logistic Regression and AdaBoost ended up close to each other overall (both around 81% accuracy), each with a slightly different trade-off between false alarms and missed churners.
- An unrestricted Decision Tree performed clearly worse, a sign it had overfit the training data rather than learning a pattern that generalises.
- Combining several differently-built models into one voting classifier is a reasonable next step to test whether their individual weaknesses cancel out — worth comparing against the individual models before settling on a final choice.
- Across every model tried, recall for the churn class stayed noticeably lower than recall for the non-churn class — a direct consequence of only about a quarter of customers actually churning, and worth addressing further (through threshold tuning, or techniques aimed specifically at class imbalance) before any of these models would be relied on operationally.